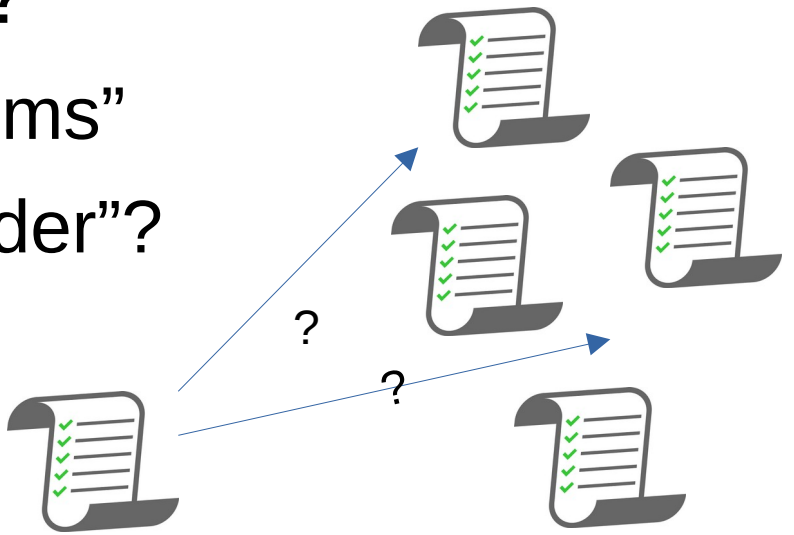
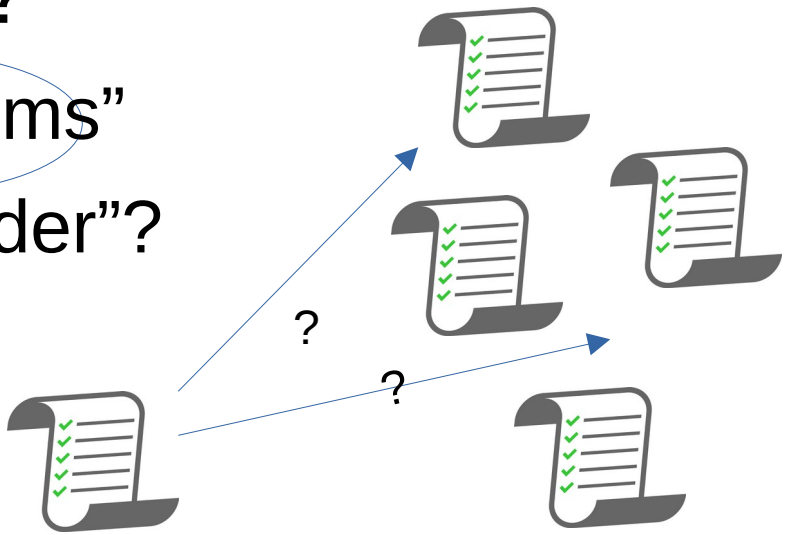
Camping List Problem

- Given 10 camping lists, how do I find the list most similar to mine?
  - Similar by “similar items”
  - Similar by “similar order”?

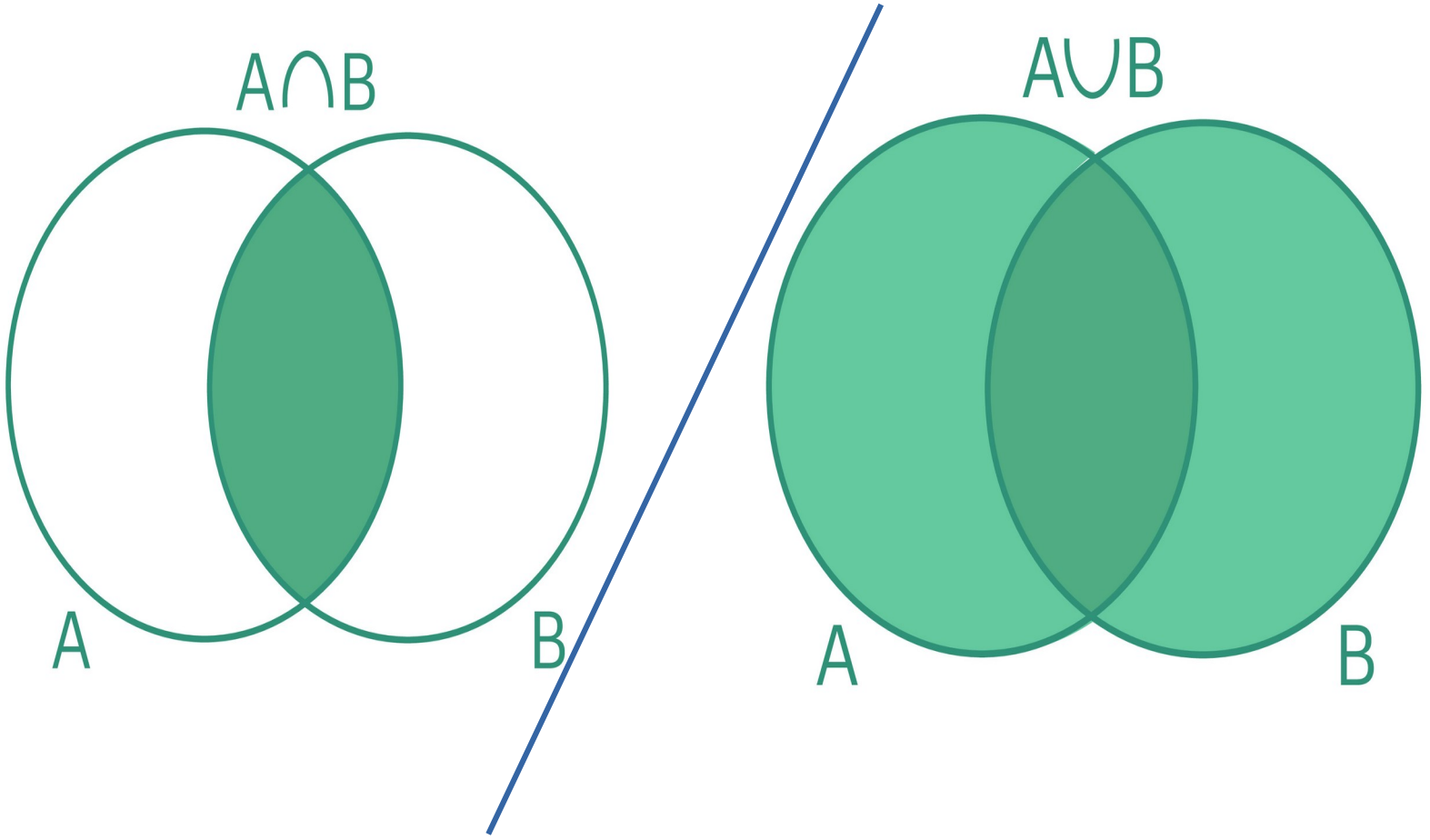


Camping List Problem

- Given 10 camping lists, how do I find the list most similar to mine?
  - Similar by “similar items”
  - Similar by “similar order”?



Jaccard Similarity



Jaccard Similarity: Camping List

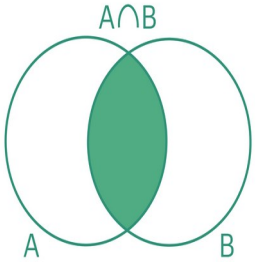
Stellan

1) Hiking Boots
2) Sunscreen
3) Bug Repellent
4) Camp Chair
5) Sleeping Bag

Beatrice

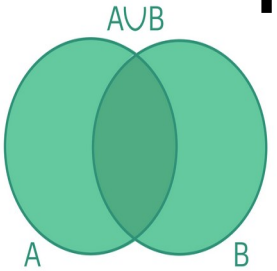
1) Sun Hat
2) Bug Repellent
3) Camp Chair
4) Tent
5) Flashlight
6) Bear spray

Jaccard Similarity: Camping List



= 2

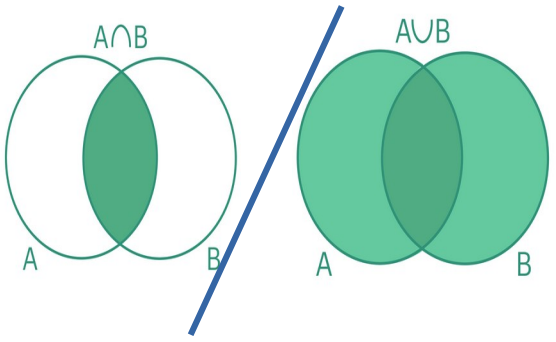
1) Bug Repellent
2) Camp Chair



= 9

1) Hiking Boots
2) Sunscreen
3) Bug Repellent
4) Sleeping Bag
5) Sun Hat
6) Camp Chair
7) Tent
8) Flashlight
9) Bear spray

Jaccard Similarity: Camping List



2 / 9 = 0.22

Jaccard Similarity: Runtime?

- How to compute intersection: O(n)
- How to compute union: O(n^2)
- How to search for the “most similar” list: O(n^3)

Is there a better Way?

- Rather than comparing items one by one, is there an efficient “signature” we can create per list?
- What trade-offs do we need to make?
  - What if we restrict the possible items on the camping list?
  - What if we loose accuracy?
- MinHash!

MinHash

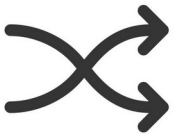
	Arno	Beatrice	Maggie	Kylian	Jesper	Jacob	Maeve	Trisha	Sandy	Zylar
h1	...									
h2										
h3										
h4										
h5										
h6										
h7										

MinHash: What is h1,h2,..?

- Answer: They are 1 signature for a given list.
- How to compute a signature?
  1) Number every camp item
  2) Shuffle the list to create a new function “h” – h1,h2..
  3) The value of “h” is

MinHash: Computing h1

1) Hiking Boots
2) Sunscreen
3) Bug Repellent
4) Sleeping Bag
5) Sun Hat
6) Camp Chair
7) Tent
8) Flashlight
9) Bear spray



1) Flashlight
2) Sleeping Bag
3) Sun Hat
4) Camp Chair
5) Sunscreen
6) Tent
7) Bear Spray
8) Hiking Boots
9) Bug Repellent

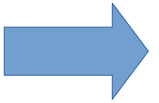
MinHash: computing h1

Shuffled list of items

1) Flashlight
2) Sleeping Bag
3) Sun Hat
4) Camp Chair
5) Sunscreen
6) Tent
7) Bear Spray
8) Hiking Boots
9) Bug Repellent

Maggie

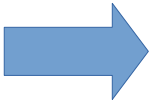
1) Bear Spray - 7
2) Sunscreen - 5
3) Camp Chair - 4
4) Tent - 6



4

Kylian

1) Camp Chair - 4
2) Flashlight - 1
3) Hiking Boots - 8
4) Tent - 6



1

MinHash

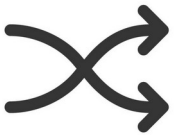
	Arno	Beatrice	Maggie	Kylian	Jesper	Jacob	Maeve	Trisha	Sandy	Zylar
h1	...		4	1						
h2										
h3										
h4										
h5										
h6										
h7										

MinHash

	Arno	Beatrice	Maggie	Kylian	Jesper	Jacob	Maeve	Trisha	Sandy	Zylar
h1	3	8	4	1	1	3	6	6	8	9
h2										
h3										
h4										
h5										
h6										
h7										

MinHash: Computing h2

1) Hiking Boots
2) Sunscreen
3) Bug Repellent
4) Sleeping Bag
5) Sun Hat
6) Camp Chair
7) Tent
8) Flashlight
9) Bear spray



1) Sun Hat
2) Camp Chair
3) Sunscreen
4) Tent
5) Bear Spray
6) Hiking Boots
7) Bug Repellent
8) Sleeping Bat
9) Flashlight

Etc...

Finally…

	Arno	Beatrice	Maggie	Kylian	Jesper	Jacob	Maeve	Trisha	Sandy	Zylar
h1	3	8	4	1	1	3	6	6	8	9
h2	5	7	9	4	3	2	4	2	8	9
h3	4	2	2	3	1	2	7	5	6	1
h4	9	7	6	5	6	3	6	2	1	7
h5	2	2	4	5	8	2	1	7	3	8
h6	9	4	9	8	6	8	3	8	5	6
h7	1	7	4	7	3	2	1	5	9	4

How do we use it?

- Lets compare Alphonse’s list to others.

  Rather than computing its Intersection and Union with all other lists, Lets compare its minHash entries..

Converting Alphonse's List

1) Bear Spray
2) Sunscreen
3) Sleeping Bag
4) Camp Chair
5) Sun Hat



h1	2
h2	5
h3	1
h4	8
h5	9
h6	1
h7	5

Now, how do we compare 2 signatures?

Alphonse

h1	2
h2	5
h3	1
h4	8
h5	9
h6	1
h7	5

vs.

Trisha

h1	3
h2	6
h3	2
h4	8
h5	9
h6	1
h7	5

Now, how do we compare 2 signatures?

Alphonse

h1	2
h2	5
h3	1
h4	8
h5	9
h6	1
h7	5

vs.

Trisha

h1	3
h2	6
h3	2
h4	8
h5	9
h6	3
h7	4

For each h_i, count if they are equal. Here we have 2 equal and 5 unequal. O(n) which is faster than O(n^2)

Our jaccard similarity is 2/5 or 0.4

Why does this work?

Theorem:

P(minhash(S) = minhash(T)) = Jaccard Similarity(S,T)

What are we supposed to do with P?

- This is why we shuffle our index list of camping items many times (here we showed 7 times)
- If two lists match their **signature** across many shufflings, there is a good probability that they are similar.
- We cannot directly find P(minhash(S) = minhash(T)). So we just count the number of times it matches...

Why did we pick our **signature**?

- Our signature is the first index that exists in the candidate list for some shuffling of items – h1,h2..
- Create 3 sets. (similar concept to U/N but regarding the entire list of possible camping items)
  - X is the items in both lists.
  - Y is the items in just 1 list.
  - Z is the items in neither list but in our full index.

What is probability that the smallest hash is equal?

- Key is that

What is the intuition?

- MinHash is a **summary statistic** across many rows.
- If two lists have many summary statistics in common, they are probably similar

Can we use MaxHash?

- Yes!
- Or SimHash – cosine similarity
- Any summary statistic
- What makes a good summary statistic?
  -

Is there a faster way?

- We can replace our summary statistic – some permutation of rows and the first index which exists in the list – with a random hash
- Random hash function is an equivalently random summary statistic
- We maintain our concept – if a random hash of the values of two lists are similar a large number of times, the lists are probably similar

Permutation vs hash function

- In a random permutation, the index of each row in our set is at a random place in the index of the universe
- The same is true of a hash function that has a range of the size of the universe – each row in our set will be some random output in the index of the universe
- The min(first encountered value) will be just as random as it was in the permutation case.

What is a random hash function

- F : (|universe|) - > (|universe|) = h (row_num) = row_num
- H: U → U =